Evacuation Pathfinding System Documentation

System Architecture, Mathematical Models, and Algorithms

August 28, 2026

Contents

1 Overview of System Components

This section provides a high-level list of the mathematical equations, algorithms, and data structures utilized in the project.

Mathematical Equations (Path Metrics):

- 2D Ground Distance (Haversine Formula)
- 3D Travel Distance (Pythagorean Theorem)
- Elevation Difference, Total Ascent, and Total Descent
- Point-to-point Slope and Overall Gradient
- Sinuosity Index (Curvature)
- Kinematic Speed Penalties (Slope and Curvature Penalties)

Algorithms:

- D* Lite (Dynamic Pathfinding Algorithm)

Data Sources & Structures:

- **Raw Data:** GPX Track Files (Spatial coordinates and elevation)
- **Processed Data:** `Calculated_Path_Metrics_GPX.csv` (Edge costs)
- **Dynamic Data:** `evacuation.db` (SQLite database storing real-time hazard reports)
- **Map Data:** Locally hosted PNG map tiles for offline viewing.

2 System Architecture and Data Flow

The following outlines how raw spatial data is processed into mathematical metrics, fed into the graph structure, and dynamically modified by real-time hazard reports before being solved by the D* Lite algorithm.

1. Raw GPX Tracks → 2. Metrics Script (Math Equations) → 3. CSV (Base Edge Costs) → 4. Graph Builder (In-Memory Nodes) → 5. D* Lite Algorithm → 6. Flask Web Interface / GeoJSON

Note: The SQLite database (Hazard Reports) dynamically feeds Severity Multipliers into Step 4 (Graph Builder).

2.1 Offline Application Architecture

The system is designed to operate entirely offline, which is critical for disaster evacuation scenarios where internet infrastructure may be compromised.

- **Local Web Server:** The Flask application (`app.py`) runs locally on the host machine.
- **Offline Mapping:** Map tiles are pre-downloaded (via `download_tiles.py`) and served directly from the `static/tiles/` directory, ensuring Leaflet maps render without internet access.
- **Local Database:** Hazard reports and historical data are managed locally using an SQLite database (`evacuation.db`), requiring no external cloud connections.
- **On-Device Computation:** All mathematical processing and D* Lite algorithm traversal happens on the local machine's CPU.

3 Detailed Explanations of Equations and Algorithms

3.1 Mathematical Equations

The system translates physical terrain into a "time cost" using the following formulas:

1. Distance Calculation

The 2D distance (d_{2d}) between two GPS points is calculated using the Haversine formula to account for the Earth's curvature.

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right) \tag{1}$$

$$d_{2d} = R \cdot 2 \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1-a}\right) \tag{2}$$

The 3D distance (d_{3d}) incorporates the elevation change (Δz) using the Pythagorean theorem:

$$d_{3d} = \sqrt{d_{2d}^2 + \Delta z^2} \tag{3}$$

2. Topographical Gradient and Sinuosity

The overall gradient determines the steepness of the path:

$$\text{Gradient} = \left(\frac{\text{Total Elevation Diff}}{d_{2d}}\right) \times 100\% \tag{4}$$

The Sinuosity Index measures how winding a road is. A perfectly straight road has an index of 1.0.

$$\text{Sinuosity} = \frac{\text{Total Path Length (along curves)}}{\text{Straight-Line Distance}} \tag{5}$$

3. Kinematic Speed Penalties

To accurately model travel time, base speed ($V_{base} = 8.33$ m/s) is penalized by steep slopes and sharp curves.

$$P_{slope} = \min\left(|\text{Gradient}| \times 0.02, 0.8\right) \tag{6}$$

$$P_{curve} = \min\left(\max(\text{Sinuosity} - 1.0, 0) \times 0.5, 0.5\right) \tag{7}$$

$$V_{adj} = V_{base} \times (1 - P_{slope}) \times (1 - P_{curve}) \tag{8}$$

$$\text{Travel Time (seconds)} = \frac{d_{3d}}{V_{adj}} \tag{9}$$

3.2 The D* Lite Algorithm

What is it?
D* Lite is an incremental heuristic search algorithm. It solves the same problem as A* or Dijkstra's (finding the shortest path from a start to a goal), but it is specifically optimized for unknown or dynamic terrains.

Why use D* Lite instead of A* or Dijkstra's?
In a standard navigation app, Dijkstra's or A* are sufficient. However, in an *evacuation scenario*, road conditions change rapidly (e.g., a landslide blocks a route, or flooding causes severe congestion). If a path is blocked while using A*, the algorithm must throw away all its previous calculations and completely recalculate the path from scratch.

D* Lite solves this by searching *backwards* from the Goal (the evacuation center) to the Start (the user). By maintaining local consistency values (`rhs` and `g` values), when a road hazard is reported via the SQLite database, D* Lite only recomputes the small localized area affected by the hazard, reusing the rest of the previously calculated graph. This makes it orders of magnitude faster at dynamic replanning.

Implementation
The `dstar_lite.py` script maintains a priority queue of inconsistent vertices. When a user submits a hazard report, `app.py` multiplies the edge cost by the hazard severity. The D* Lite instance detects this edge cost change, places the affected nodes back into the priority queue, and efficiently ripples the changes through the graph to find the new optimal path.

4 Recommendations and Missing Factors

While the current model effectively translates distance, elevation, and curvature into travel time, the following factors are currently lacking and represent opportunities for future enhancement:

1. **Road Surface Condition:** The CSV contains an empty `Condition` column. A dirt or muddy road will drastically reduce the friction coefficient and safe travel speed compared to paved concrete. Implementing a surface multiplier (e.g., $V_{adj} \times 0.6$ for dirt) is highly recommended.

2. **Road Width and Type:** The `Road_Type` column is empty. Narrow single-lane barangay roads restrict speed compared to multi-lane highways, especially during high-volume evacuations where opposing traffic or bottlenecks occur.

3. **Weather Dynamics:** Extreme weather (typhoons, heavy rain) reduces visibility and tire traction. A global system variable to penalize all edge speeds during a weather event would increase realism.

4. **Intersection/Node Penalties:** Currently, the time cost is strictly edge-based. The model does not penalize routes that cross multiple intersections, which practically require vehicles to stop, yield, and accelerate. Adding a flat time penalty for every node traversed would favor bypass highways over dense city blocks.

5. **Traffic Congestion Simulation:** While the hazard reports act as a proxy for congestion, the base model assumes the evacuee is the only vehicle on the road. Evacuations inherently cause traffic jams. Integrating a dynamic flow model where edge cost increases as more users are routed through the same edge would create a load-balancing evacuation system.